

College AC Management System

Backend design document. How CSV uploads become trustworthy database records — parsing, validation, staging, reconciliation, and reporting — without ever destroying data that is already there.

Database

PostgreSQL 14+

Pattern

Stage & reconcile

Entities

7 core + 2 staging

Revision

1.0

01 What this system does

A college owns hundreds of air conditioners spread across blocks, floors, departments and rooms. Management tracks them in spreadsheets. The backend's job is to turn those spreadsheets into one reliable database — and to keep doing that every time a new spreadsheet arrives, for years.

The hard part is not the first upload. It is the fifth one. Someone uploads 370 ACs in January, 100 more in March, then accidentally uploads the March file again in April, then in June uploads a file that only has two columns because they only wanted to change the status of a few units.

A naive importer breaks on every one of those. This design handles all of them through a single rule set:

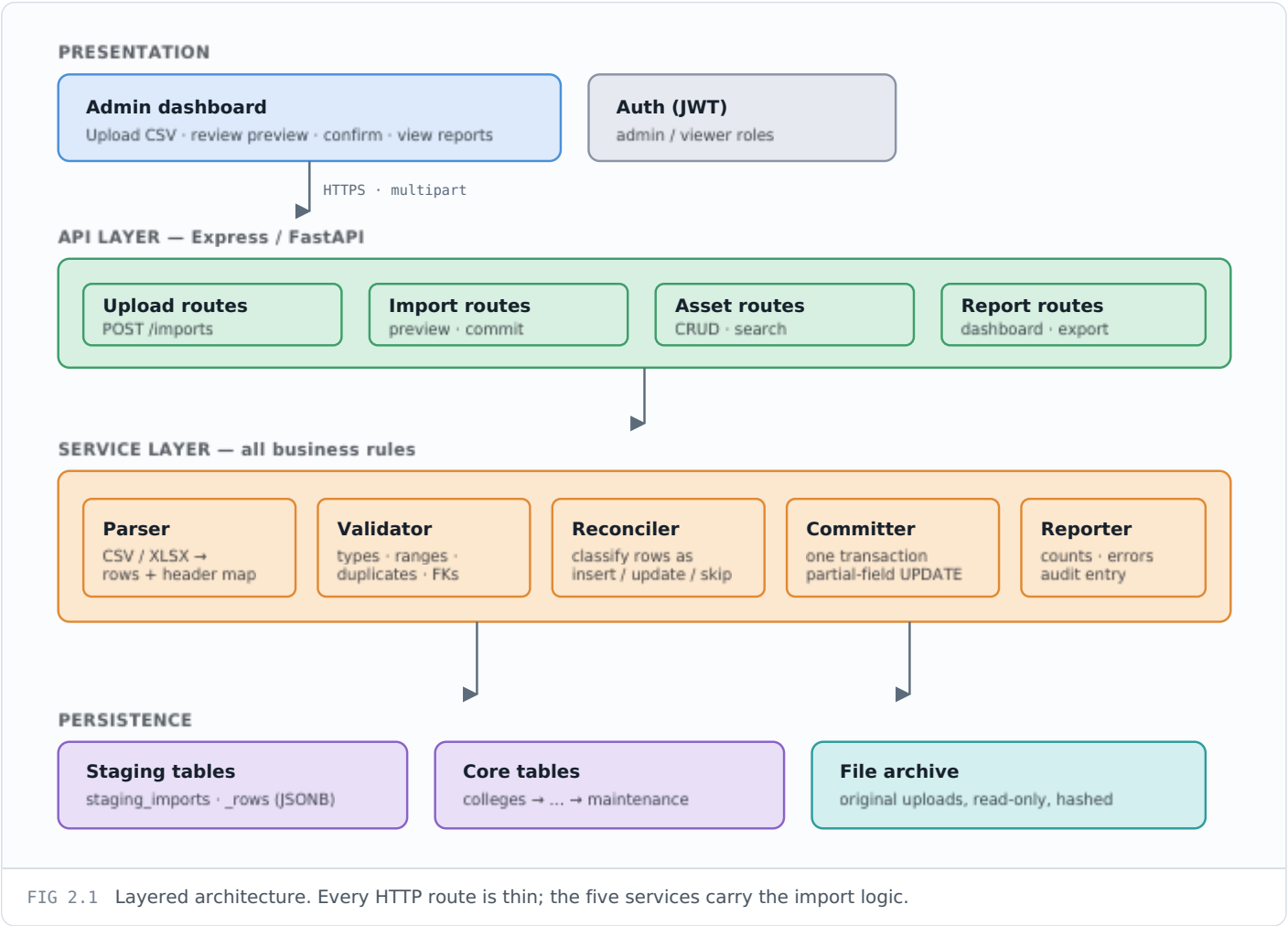
- **Never replace, always merge.** An upload adds to and corrects the database. It does not define it.
- **Identity decides everything.** `ac_id` tells the system whether a row is new or known.
- **Absent is not empty.** A column missing from the CSV means "no opinion", not "set to null".
- **Nothing changes until it is checked.** Rows land in staging first and are only committed after validation passes.
- **Every import is explainable.** The admin sees exactly what was inserted, updated, skipped and rejected.

Core principle

PostgreSQL is the permanent source of truth. CSV and Excel files are import sources, kept afterwards only as archived evidence of what was submitted and by whom.

02 System architecture

Four layers, each with one responsibility. The service layer is where all the intelligence lives; the API layer only translates HTTP into service calls, and the data layer only stores.



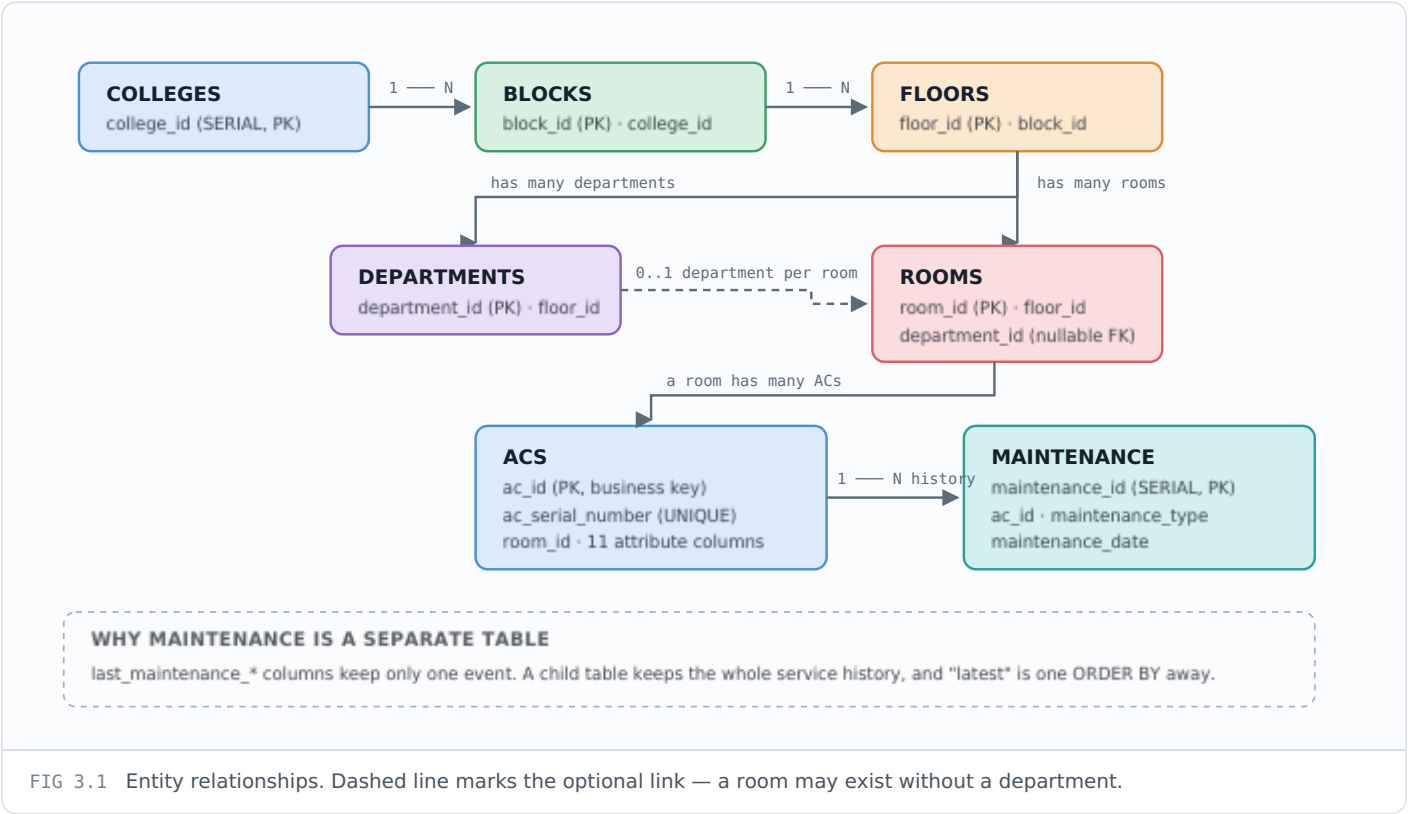
Technology choices

Concern	Choice	Why
Database	PostgreSQL 14+	Strong constraints, <code>ON CONFLICT</code> upserts, JSONB for staging rows, real transactions.
Runtime	Node.js + Express, or Python + FastAPI	Either works. Examples below use Node with <code>pg</code> ; the SQL is identical in Python.
CSV parsing	<code>csv-parse</code> / <code>pandas</code>	Streaming, quoted-field safe, does not load 10 000 rows into memory at once.
Excel parsing	<code>SheetJS</code> / <code>openpyxl</code>	Management will send <code>.xlsx</code> eventually. Normalise it to the same row shape as CSV.
File storage	Local disk or S3-compatible bucket	Originals are kept for audit; the database stores only the path and SHA-256 hash.

03Data model

The physical hierarchy of the campus is modelled directly. Each level owns the one below it,

so any AC can be traced back to its room, floor, block and college with joins alone.



Key design decisions

Decision	Reasoning
ac_id is a VARCHAR business key, not a serial	Management already writes AC-001 on the unit and in the sheet. Matching on the same string they use is what makes reconciliation possible at all.
ac_serial_number is UNIQUE	A second identity check. Catches the case where the same physical unit is entered twice under two different IDs.
department_id on rooms is nullable	Corridors, auditoriums and common areas have ACs but belong to no department.
number_of_acs is optional	It is a cached count. The true value is always SELECT COUNT(*) FROM acs WHERE room_id = ?. Refresh it with a trigger or drop it.
ON DELETE RESTRICT everywhere	Deleting a block should never silently take 80 ACs with it. The database refuses; the admin must deal with children explicitly.
Maintenance in its own table	Full service history per AC, which is what a maintenance system is actually for.

04Core schema SQL

Create the tables in this order — each depends on the one before it. A few additions to the original script are marked in comments.

```
-- =====
-- 1. COLLEGES
-- =====
```

```

CREATE TABLE colleges (
    college_id SERIAL PRIMARY KEY,
    college_name VARCHAR(255) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- 2. BLOCKS
CREATE TABLE blocks (
    block_id VARCHAR(50) PRIMARY KEY,
    college_id INT NOT NULL REFERENCES colleges(college_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    block_name VARCHAR(255) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- 3. FLOORS
CREATE TABLE floors (
    floor_id VARCHAR(50) PRIMARY KEY,
    block_id VARCHAR(50) NOT NULL REFERENCES blocks(block_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    floor_name VARCHAR(100) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- 4. DEPARTMENTS
CREATE TABLE departments (
    department_id VARCHAR(50) PRIMARY KEY,
    floor_id VARCHAR(50) NOT NULL REFERENCES floors(floor_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    department_name VARCHAR(255) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- 5. ROOMS
CREATE TABLE rooms (
    room_id VARCHAR(50) PRIMARY KEY,
    floor_id VARCHAR(50) NOT NULL REFERENCES floors(floor_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    department_id VARCHAR(50) REFERENCES departments(department_id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    room_name VARCHAR(255) NOT NULL,
    room_type VARCHAR(100),
    number_of_acs INT DEFAULT 0 CHECK (number_of_acs >= 0),
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- 6. ACS (the table the CSV import writes to)
CREATE TABLE acs (
    ac_id VARCHAR(50) PRIMARY KEY,
    room_id VARCHAR(50) NOT NULL REFERENCES rooms(room_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    ac_make_company VARCHAR(255),
    ac_model_number VARCHAR(100),
    ac_serial_number VARCHAR(100) UNIQUE,
    ac_capacity_ton DECIMAL(4,2) CHECK (ac_capacity_ton IS NULL OR ac_capacity_ton > 0),
    ac_type VARCHAR(50),
    manufacturing_year INT CHECK (manufacturing_year IS NULL OR manufacturing_year BETWEEN 1900 AND 2100),
    installation_year INT CHECK (installation_year IS NULL OR installation_year BETWEEN 1900 AND 2100),
    current_status VARCHAR(50) DEFAULT 'Active',
    ac_star_rating INT CHECK (ac_star_rating IS NULL OR ac_star_rating BETWEEN 1 AND 5),
    power_rating_kw DECIMAL(5,2) CHECK (power_rating_kw IS NULL OR power_rating_kw >= 0),

```

```

power_factor          DECIMAL(4,2) CHECK (power_factor IS NULL OR power_factor BETWEEN 0 AND 1),

-- ADDED: soft removal instead of DELETE (see section 11)
is_active             BOOLEAN NOT NULL DEFAULT TRUE,
decommissioned_at     TIMESTAMPTZ,

created_at            TIMESTAMPTZ DEFAULT now(),
updated_at            TIMESTAMPTZ DEFAULT now(),

-- ADDED: install year cannot precede manufacture year
CONSTRAINT chk_year_order CHECK (
    manufacturing_year IS NULL OR installation_year IS NULL
    OR installation_year >= manufacturing_year
)
);

-- 7. MAINTENANCE
CREATE TABLE maintenance (
    maintenance_id     SERIAL PRIMARY KEY,
    ac_id               VARCHAR(50) NOT NULL REFERENCES acs(ac_id)
                        ON UPDATE CASCADE ON DELETE RESTRICT,
    maintenance_type    VARCHAR(100) NOT NULL,
    maintenance_date    DATE NOT NULL,
    remarks             TEXT,
    created_at          TIMESTAMPTZ DEFAULT now()
);

-- =====
-- INDEXES
-- =====
CREATE INDEX idx_blocks_college      ON blocks(college_id);
CREATE INDEX idx_floors_block        ON floors(block_id);
CREATE INDEX idx_departments_floor   ON departments(floor_id);
CREATE INDEX idx_rooms_floor         ON rooms(floor_id);
CREATE INDEX idx_rooms_department    ON rooms(department_id);
CREATE INDEX idx_acs_room            ON acs(room_id);
CREATE INDEX idx_acs_status          ON acs(current_status);
CREATE INDEX idx_maintenance_ac      ON maintenance(ac_id);
CREATE INDEX idx_maintenance_date    ON maintenance(maintenance_date DESC);

-- keep updated_at honest
CREATE OR REPLACE FUNCTION touch_updated_at() RETURNS trigger AS $$
BEGIN
    NEW.updated_at := now();
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_acs_touch BEFORE UPDATE ON acs
FOR EACH ROW EXECUTE FUNCTION touch_updated_at();

```

On TIMESTAMPTZ

Plain `TIMESTAMP` loses the time zone, which becomes a problem the moment the server and the college are configured differently. `TIMESTAMPTZ` costs nothing extra and stores an unambiguous instant.

05 Staging tables

Staging is the buffer between an uploaded file and the real database. Every row of every upload lands here first, keeps its original line number, and carries its own validation verdict.

Nothing touches `acs` until the whole file has been judged.

The important trick is `raw_data JSONB`. Storing the row as a JSON object preserves exactly which keys the CSV supplied — which is the only way to distinguish "the column was absent" from "the value was blank". A fixed set of staging columns would erase that difference immediately.

```
-- one row per uploaded file
CREATE TABLE staging_imports (
  import_id      SERIAL PRIMARY KEY,
  file_name      VARCHAR(255) NOT NULL,
  file_path      TEXT          NOT NULL,    -- archived original
  file_hash      CHAR(64)      NOT NULL,    -- SHA-256, detects re-uploads
  target_entity  VARCHAR(50)   NOT NULL,    -- 'acs' | 'rooms' | 'maintenance'
  detected_columns TEXT[]      NOT NULL,    -- header row, normalised

  total_rows     INT DEFAULT 0,
  valid_rows     INT DEFAULT 0,
  invalid_rows   INT DEFAULT 0,
  inserted_rows  INT DEFAULT 0,
  updated_rows   INT DEFAULT 0,
  unchanged_rows INT DEFAULT 0,
  duplicate_rows INT DEFAULT 0,

  clear_empty_fields BOOLEAN NOT NULL DEFAULT FALSE, -- import option
  allow_insert      BOOLEAN NOT NULL DEFAULT TRUE,
  allow_update      BOOLEAN NOT NULL DEFAULT TRUE,

  status         VARCHAR(20) NOT NULL DEFAULT 'uploaded',
  -- uploaded → parsed → validated → previewed → committed | failed | cancelled

  uploaded_by     INT,
  uploaded_at     TIMESTAMPTZ DEFAULT now(),
  committed_at    TIMESTAMPTZ,
  error_message    TEXT
);

-- one row per CSV line
CREATE TABLE staging_import_rows (
  row_id         BIGSERIAL PRIMARY KEY,
  import_id      INT NOT NULL REFERENCES staging_imports(import_id) ON DELETE CASCADE,
  line_number     INT NOT NULL,             -- CSV line, for error messages
  business_key    VARCHAR(100),             -- ac_id extracted from the row
  raw_data        JSONB NOT NULL,           -- ONLY the keys the CSV supplied

  action          VARCHAR(20),              -- insert | update | unchanged | duplicate | reject
  changed_fields  TEXT[],                   -- what an update would actually change
  errors          JSONB DEFAULT '[]'::jsonb,
  created_at      TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_srows_import ON staging_import_rows(import_id);
CREATE INDEX idx_srows_key    ON staging_import_rows(business_key);
CREATE INDEX idx_srows_action ON staging_import_rows(import_id, action);
CREATE UNIQUE INDEX idx_imports_hash ON staging_imports(file_hash)
  WHERE status = 'committed'; -- flag identical re-uploads
```

Why not a typed staging table

If staging had a fixed `power_factor DECIMAL` column, an absent CSV column and an empty CSV cell would both arrive as `NULL`. The distinction would be gone before reconciliation ever ran. JSONB keeps the shape of what was actually submitted.

06 The import pipeline

This is the heart of the backend. An upload passes through seven stages and can only fail forward into a report — never into a half-written database.

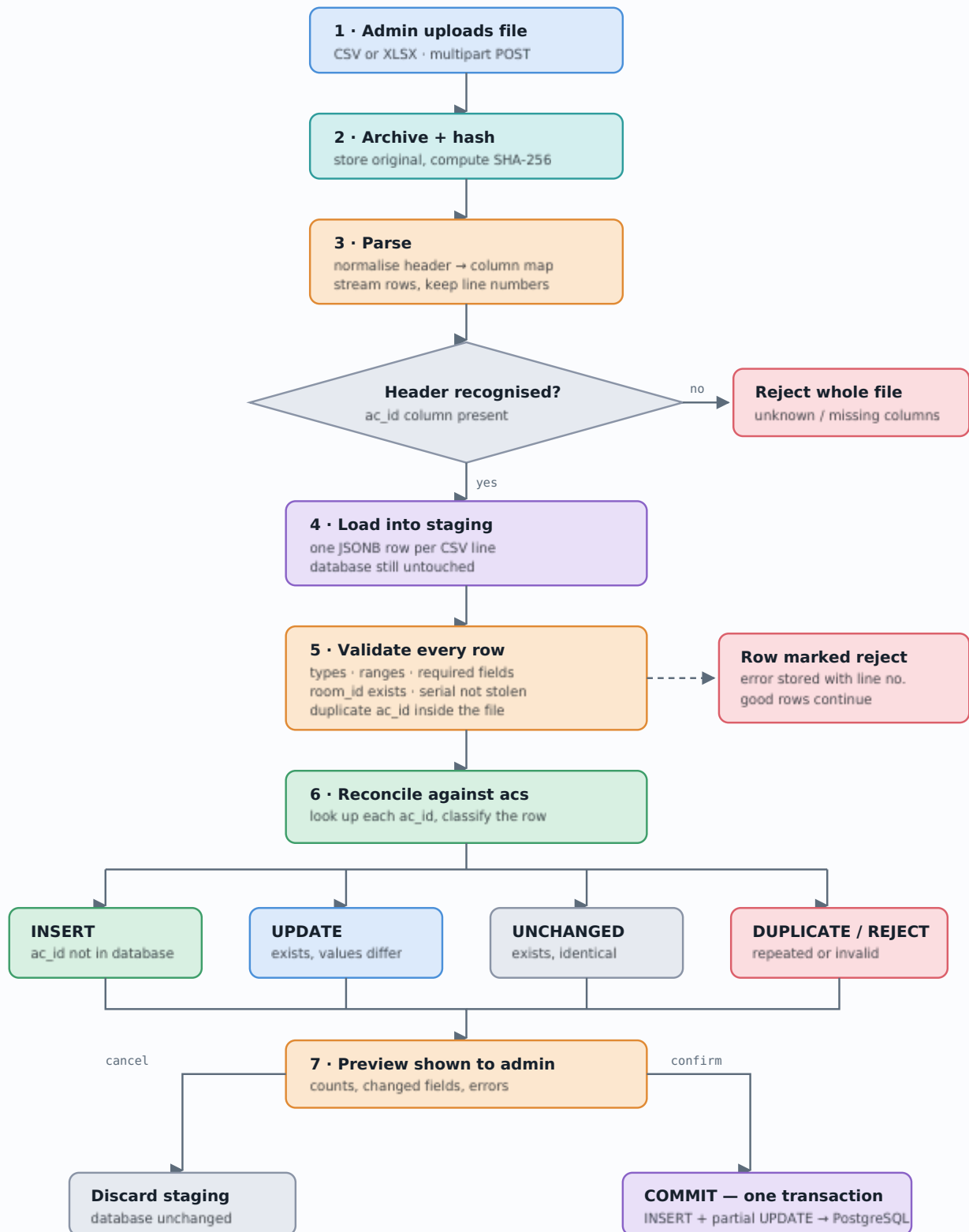


FIG 6.1 The seven-stage import pipeline. The only path that writes to `acs` is an explicit admin confirmation at stage 7.

07 Stage by stage

Stage 1-2 • Upload and archive

The file is written to storage before anything else happens, and a SHA-256 hash is taken. The hash gives you duplicate-upload detection for free: if a committed import already has the same hash, you can warn the admin that this exact file has been processed before. Limits worth enforcing here: maximum file size, allowed extensions (`.csv` , `.xlsx`), and MIME sniffing rather than trusting the extension.

Stage 3 • Parse and map the header

Header names arrive inconsistent — `AC_ID` , `ac id` , `"AC ID "` . Normalise aggressively, then map through an alias table so that management does not have to memorise exact column spellings.

```
const COLUMN_ALIASES = {
  ac_id: ['ac_id', 'acid', 'ac_no', 'ac_number', 'asset_id'],
  room_id: ['room_id', 'room', 'room_no'],
  ac_make_company: ['ac_make_company', 'company', 'make', 'brand', 'manufacturer'],
  ac_model_number: ['ac_model_number', 'model', 'model_no'],
  ac_serial_number: ['ac_serial_number', 'serial', 'serial_no', 'sl_no'],
  ac_capacity_ton: ['ac_capacity_ton', 'capacity', 'capacity_ton', 'tonnage', 'ton'],
  ac_type: ['ac_type', 'type'],
  manufacturing_year: ['manufacturing_year', 'mfg_year', 'year_of_manufacture'],
  installation_year: ['installation_year', 'install_year', 'year_installed'],
  current_status: ['current_status', 'status', 'condition'],
  ac_star_rating: ['ac_star_rating', 'star', 'star_rating', 'rating'],
  power_rating_kw: ['power_rating_kw', 'power', 'power_kw', 'kw'],
  power_factor: ['power_factor', 'pf']
};

const normalise = h => h.trim().toLowerCase()
  .replace(/[\\s\\-\\.]+/g, '_').replace(/^[^a-z0-9_]/g, '');

function mapHeader(headerRow) {
  const map = {}, unknown = [];
  headerRow.forEach((raw, idx) => {
    const n = normalise(raw);
    const hit = Object.entries(COLUMN_ALIASES)
      .find(([aliases]) => aliases.includes(n));
    hit ? map[hit[0]] = idx : unknown.push(raw);
  });
  if (!('ac_id' in map)) throw new ImportError('E_NO_KEY',
    'The file must contain an AC ID column.');
```

```
  return { map, unknown, suppliedFields: Object.keys(map) };
}
```

`suppliedFields` is the list that section 09 depends on. Carry it on the import record as `detected_columns` .

Stage 4 • Load into staging

Each parsed row becomes one JSONB object containing only the keys the file supplied. Insert in batches of 500–1000 rather than one statement per row.

```
function rowToJson(values, map) {
  const obj = {};
  for (const [field, idx] of Object.entries(map)) {
    obj[field] = (values[idx] ?? '').trim(); // key present even if blank
  }
  return obj; // absent columns simply have no key
}
```

Stage 5 • Validate

Validation is per row and never aborts the file. A bad row is marked `reject` with a readable message and a line number; the rest of the file still gets imported. See section 14 for the full rule table.

Stage 6 • Reconcile

One query fetches every existing AC mentioned in the file, then each staged row is classified in memory. This is a single round trip, not one lookup per row.

```
-- pull every relevant existing record in one shot
SELECT * FROM acs
WHERE ac_id = ANY($1::varchar[]);
```

Stage 7 • Preview, then commit

The preview endpoint returns the counts and a sample of changes. Only when the admin confirms does the committer open a transaction and apply the staged rows. This separation is what makes a mistaken upload harmless.

08 Reconciliation logic

Reconciliation answers one question per row: *does this AC already exist, and if so, does anything actually change?*

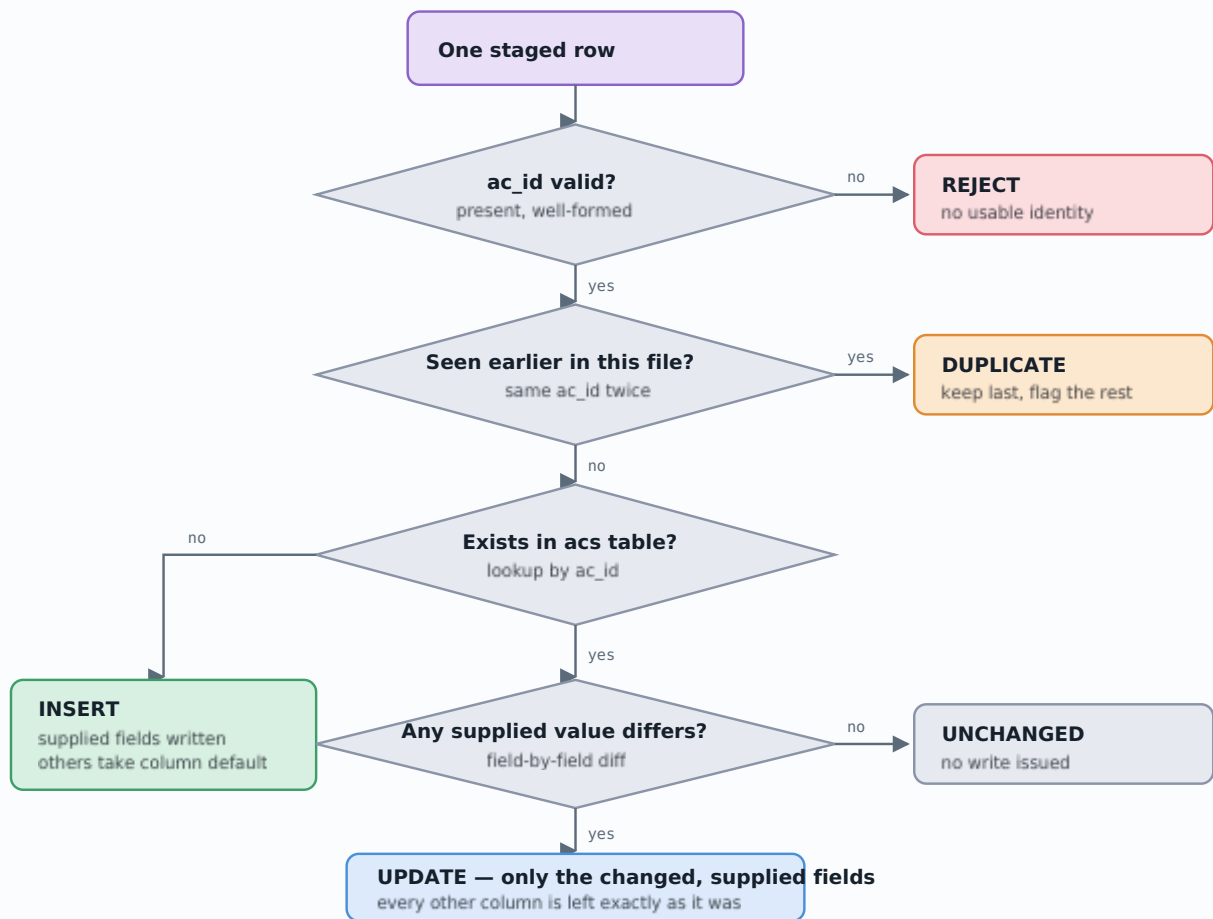


FIG 8.1 Per-row decision tree. Five terminal outcomes, each counted separately in the import report.

Worked example

Database holds 370 ACs. A file with 100 rows arrives.

Row	ac_id	Lookup	Outcome
1	AC-371	not found	INSERT
2	AC-372	not found	INSERT
...	...	...	...
95	AC-045	found, status differs	UPDATE
96	AC-046	found, identical	UNCHANGED
99	AC-371	already in this file	DUPLICATE
100	blank	—	REJECT

Result: 370 + 94 inserts = **464 ACs**, with 4 updated, 1 unchanged, 1 duplicate and 1 rejected. Nothing was deleted and no existing record lost a field.

The commit statement

A single upsert handles insert-or-update, and `COALESCE` on the excluded value protects existing data when the CSV sends a blank.

```

INSERT INTO acs (ac_id, room_id, ac_make_company, current_status, power_rating_kw)
VALUES ($1, $2, $3, $4, $5)
ON CONFLICT (ac_id) DO UPDATE SET
    room_id      = COALESCE(EXCLUDED.room_id,      acs.room_id),
    ac_make_company = COALESCE(EXCLUDED.ac_make_company, acs.ac_make_company),
    current_status = COALESCE(EXCLUDED.current_status, acs.current_status),
    power_rating_kw = COALESCE(EXCLUDED.power_rating_kw, acs.power_rating_kw),
    updated_at    = now()
RETURNING (xmax = 0) AS was_inserted;  -- true = INSERT, false = UPDATE

```

Reading the result

`xmax = 0` is the standard way to tell whether an upsert inserted or updated. Use it to build the import counts without a second query.

The statement above still lists every column, which is fine when the CSV is full-width. For partial files, build the column list dynamically from `suppliedFields` — that is the subject of the next section.

09 Missing fields — the three cases

A CSV should never be required to carry every database column. Management should be able to send a two-column file that changes only the status of six units, and everything else must survive untouched.

Three situations look similar and must be treated differently.



The rule that protects the data

A column missing from the CSV is never a request to delete. Two columns supplied means two columns updated — the other twelve keep whatever they had.

Building the UPDATE from supplied fields only

The `SET` clause is generated per import from `detected_columns`, so a two-column file produces a two-column update.

```
function buildUpsert(suppliedFields, { clearEmptyFields }) {
  // ac_id is the conflict target, never in the SET list
  const cols = suppliedFields.filter(f => f !== 'ac_id');

  const setClause = cols.map(c => clearEmptyFields
    ? `${c} = EXCLUDED.${c}` // blank → NULL
    : `${c} = COALESCE(EXCLUDED.${c}, acs.${c})` // blank → keep old
  ).join(',\n    ');

  return `
  INSERT INTO acs (ac_id, ${cols.join(', ')})
  VALUES ($1, ${cols.map(_, i) => '$' + (i + 2)}.join(', '))
  ON CONFLICT (ac_id) DO UPDATE SET
  ${setClause},
  updated_at = now()
  RETURNING (xmax = 0) AS was_inserted`;
}
```

One detail that matters

Convert empty strings to `NULL` before binding parameters. `COALESCE` treats `''` as a real value and will happily overwrite `Daikin` with an empty string.

```
const clean = v => (v === undefined || v === null || String(v).trim() === '')
  ? null : String(v).trim();
```

Import options offered to the admin

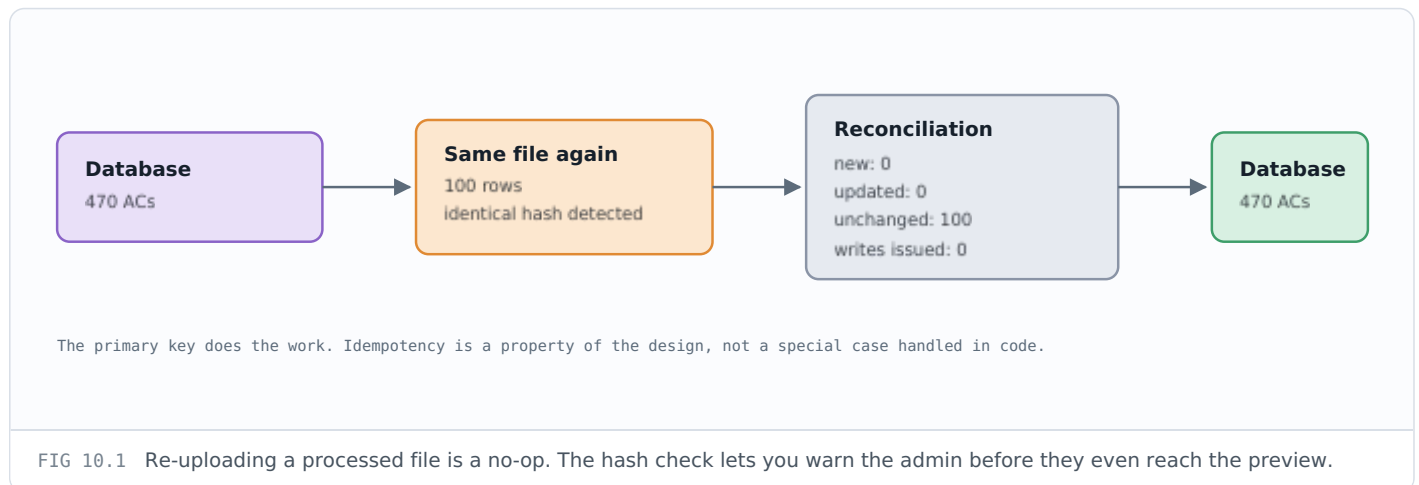
Option	Default	Effect
Add new records	on	Rows whose <code>ac_id</code> is unknown are inserted. Turn off to run a corrections-only import.
Update existing records	on	Known rows are updated. Turn off to run an additions-only import.
Clear existing values when the CSV cell is empty	off	Case 3 above. Destructive; record who enabled it and when.

10 Re-uploading the same file

Someone will upload the March file twice. The database must still hold 470 ACs afterwards, not 570.

It does, automatically, because reconciliation matches on `ac_id`. The second pass finds

every row already present and identical, so all 100 rows land in `UNCHANGED` and no write is issued at all.



11 Deletion policy

This is where a careless importer destroys a year of records. A file with 100 rows does not mean the other 370 ACs were scrapped — it almost always means someone exported only the new purchases.

Absolute rule

Missing from the CSV is never a reason to delete. An import only ever inserts and updates.

Removal is a separate, deliberate admin action, and even then it should be a soft removal:

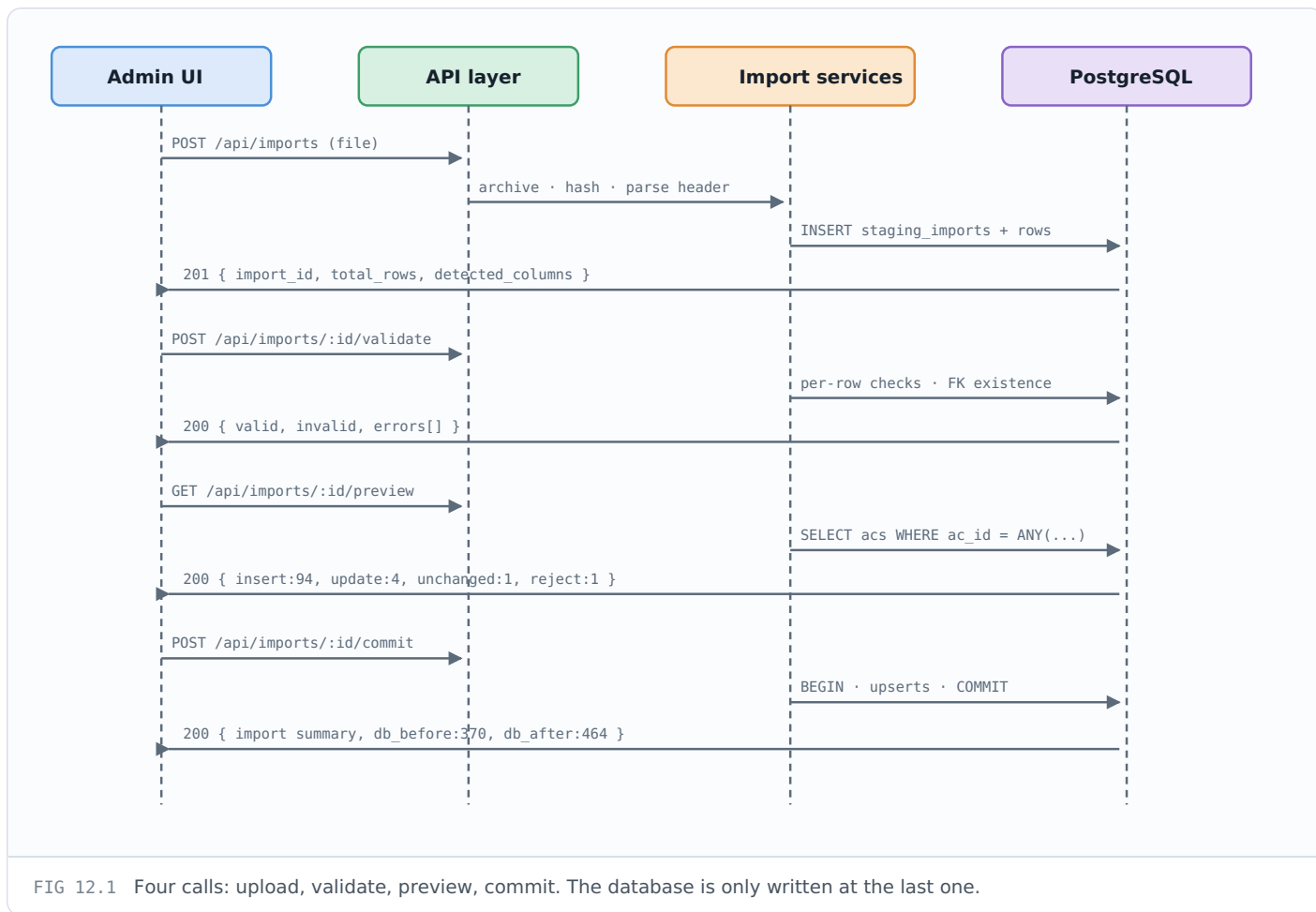
```
UPDATE acs
SET   current_status = 'Decommissioned',
      is_active       = FALSE,
      decommissioned_at = now(),
      updated_at       = now()
WHERE ac_id = $1;
```

Keeping the row preserves the maintenance history attached to it, keeps historical reports accurate, and means an AC that was removed by mistake can be restored. Dashboards simply filter on `is_active = TRUE`.

If a genuine full-replacement import is ever needed — a complete re-audit of the campus, say — make it an explicit, separately-named operation that first shows the admin exactly which ACs would be marked missing, and never let it be the default behaviour of a normal upload.

12 Request sequence

How the four calls of a full import interleave across the stack.



13 API reference

Import endpoints

Method & path	Purpose	Notes
POST /api/imports	Upload a file, parse it, load staging	multipart; body carries <code>target_entity</code> . Returns <code>import_id</code> .
POST /api/imports/:id/validate	Run all row checks	Writes verdicts into <code>staging_import_rows.errors</code> .
GET /api/imports/:id/preview	Classified counts + sample diffs	Read-only. Safe to call repeatedly.
POST /api/imports/:id/commit	Apply staged rows	Body carries the three import options. Transactional.
DELETE /api/imports/:id	Cancel and discard staging	Only allowed before commit.
GET /api/imports	Import history	Paginated; who uploaded what, and when.
GET /api/imports/:id/errors.csv	Download rejected rows	Admin fixes the file and re-uploads only the failures.

Asset endpoints

Method & path	Purpose
GET /api/acs	List with filters: ?status= ?block_id= ?room_id= ?q= , paginated
GET /api/acs/:ac_id	One AC with its location chain and maintenance history
PATCH /api/acs/:ac_id	Single-record edit from the UI
POST /api/acs/:ac_id/decommission	Soft removal (section 11)
POST /api/acs/:ac_id/maintenance	Log a service event
GET /api/dashboard/summary	Counts by status, block, company, star rating; total connected load

Response shape for a preview

```
{
  "import_id": 12,
  "file_name": "new_acs_100.csv",
  "detected_columns": ["ac_id", "ac_make_company", "current_status"],
  "ignored_columns": ["Remarks"],
  "counts": {
    "total": 100, "insert": 94, "update": 4,
    "unchanged": 1, "duplicate": 1, "reject": 1
  },
  "db_before": 370,
  "db_after_if_committed": 464,
  "sample_updates": [
    { "ac_id": "AC-045",
      "changes": { "current_status": ["Active", "Under Maintenance"] } }
  ],
  "errors": [
    { "line": 100, "code": "E_MISSING_KEY", "message": "AC ID is blank." }
  ]
}
```

14Validation rules

Run these per row during stage 5. Each failure records a code, a message written for the person who made the spreadsheet, and the line number.

Code	Check	Message shown to the admin
E_MISSING_KEY	ac_id present and non-blank	AC ID is blank.
E_KEY_FORMAT	Matches <code>^[A-Za-z0-9\-_\/]{1,50}\$</code>	AC ID contains characters that are not allowed.
E_DUP_IN_FILE	Same ac_id appears once in the file	This AC ID appears on line 12 as well.
E_ROOM_UNKNOWN	room_id exists in rooms	Room R-204 is not in the system. Add the room first.
E_ROOM_REQUIRED	New ACs must name a room	A new AC needs a room ID.
E_SERIAL_TAKEN	Serial not already used by a different ac_id	Serial DK12345 already belongs to AC-118.
E_NUMBER	Numeric fields parse as numbers	Capacity "1.5 Ton" is not a number. Use 1.5.
E_RANGE_STAR	Star rating 1-5	Star rating must be between 1 and 5.
E_RANGE_PF	Power factor 0-1	Power factor must be between 0 and 1.
E_RANGE_YEAR	Years 1900-2100, install ≥ manufacture	Installation year is before the manufacturing year.
E_ENUM_STATUS	Status is one of the allowed values	Status "Broken" is not recognised. Use Active, Under Maintenance, Not Working or Decommissioned.
E_LENGTH	Text fits the column width	Model number is longer than 100 characters.

Normalisation before validation

Half of the errors above disappear if you clean values first — and cleaning is safer than rejecting for things that are unambiguous.

- Trim whitespace, collapse internal doubles, strip a UTF-8 BOM from the first header cell.
- "1.5 Ton" , "1.5T" , "1,5" → 1.5 .
- "active" , "ACTIVE" , "Working" → Active , through a status alias map.
- Excel date cells → ISO YYYY-MM-DD before they reach a DATE column.
- Excel's habit of turning AC-001 into a formula or a number is worth a specific check.

Status values

Keep the allowed list in a lookup table rather than a CHECK constraint, so an admin can add a new status without a migration.

15Transactions and concurrency

The commit is all-or-nothing. If row 400 of 500 violates a constraint that validation somehow missed, the whole import rolls back and the database is exactly as it was before the admin clicked confirm.

```

async function commitImport(importId, options) {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');

    const before = await countAcs(client);
    const rows = await fetchStagedRows(client, importId, ['insert', 'update']);
    const sql = buildUpsert(await suppliedFields(client, importId), options);

    let inserted = 0, updated = 0;
    for (const batch of chunk(rows, 500)) {
      for (const r of batch) {
        const res = await client.query(sql, toParams(r));
        res.rows[0].was_inserted ? inserted++ : updated++;
      }
    }

    const after = await countAcs(client);
    await client.query(
      `UPDATE staging_imports
       SET status = 'committed', committed_at = now(),
         inserted_rows = $2, updated_rows = $3
       WHERE import_id = $1`, [importId, inserted, updated]);

    await client.query('COMMIT');
    return { inserted, updated, before, after };

  } catch (err) {
    await client.query('ROLLBACK');
    await markImportFailed(importId, err); // separate connection
    throw err;
  } finally {
    client.release();
  }
}

```

Two admins, one moment

If two people commit overlapping imports at the same time, the later one wins on any shared `ac_id` — `ON CONFLICT DO UPDATE` serialises them safely, no row is lost, and neither transaction fails. If you need stricter behaviour, take an advisory lock per `target_entity` for the duration of the commit so imports queue rather than interleave.

Performance notes

- Row-by-row upserts are fine to about 10 000 rows. Beyond that, `COPY` into a temporary table and run one set-based `INSERT ... SELECT ... ON CONFLICT`.
- For files above a few thousand rows, run validation and commit as a background job and let the UI poll `GET /api/imports/:id` for status.
- Reconciliation must never issue one `SELECT` per row. One `= ANY($1::varchar[])` query, then classify in memory.

16 Project structure

The shape below keeps route handlers thin and puts every import rule in one folder, which is where you will spend most of your time.

```

backend/
├─ src/
│  ├─ config/
│  │  ├─ db.js                pg Pool, single shared instance
│  │  └─ env.js              validated environment variables
│  ├─ routes/
│  │  ├─ import.routes.js    upload · validate · preview · commit
│  │  ├─ ac.routes.js
│  │  ├─ location.routes.js  colleges · blocks · floors · rooms
│  │  └─ report.routes.js
│  ├─ controllers/          HTTP in, service call, HTTP out
│  │  ├─ import.controller.js
│  │  └─ ac.controller.js
│  ├─ services/             all business rules live here
│  │  ├─ import/
│  │  │  ├─ parser.service.js  CSV / XLSX → rows
│  │  │  ├─ headerMap.js      alias table + normalisation
│  │  │  ├─ staging.service.js load into staging tables
│  │  │  ├─ validator.service.js per-row rules, error codes
│  │  │  ├─ reconciler.service.js insert / update / unchanged
│  │  │  ├─ committer.service.js transaction + dynamic upsert
│  │  │  └─ report.service.js  counts, error CSV export
│  │  └─ ac.service.js
│  │  └─ dashboard.service.js
│  ├─ repositories/        raw SQL, nothing else
│  │  ├─ ac.repo.js
│  │  ├─ room.repo.js
│  │  └─ staging.repo.js
│  ├─ middleware/
│  │  ├─ auth.js            JWT, role check
│  │  ├─ upload.js          multer, size and MIME limits
│  │  └─ errorHandler.js    maps ImportError → HTTP
│  ├─ utils/
│  │  ├─ clean.js           trim, empty → null, number parsing
│  │  └─ hash.js            SHA-256 of uploaded file
│  └─ app.js
├─ migrations/
│  ├─ 001_core_schema.sql
│  ├─ 002_staging_tables.sql
│  └─ 003_audit_log.sql
├─ storage/uploads/        archived originals, read-only
├─ tests/
│  ├─ reconciler.test.js
│  ├─ partialUpdate.test.js
│  └─ fixtures/*.csv
└─ package.json

```

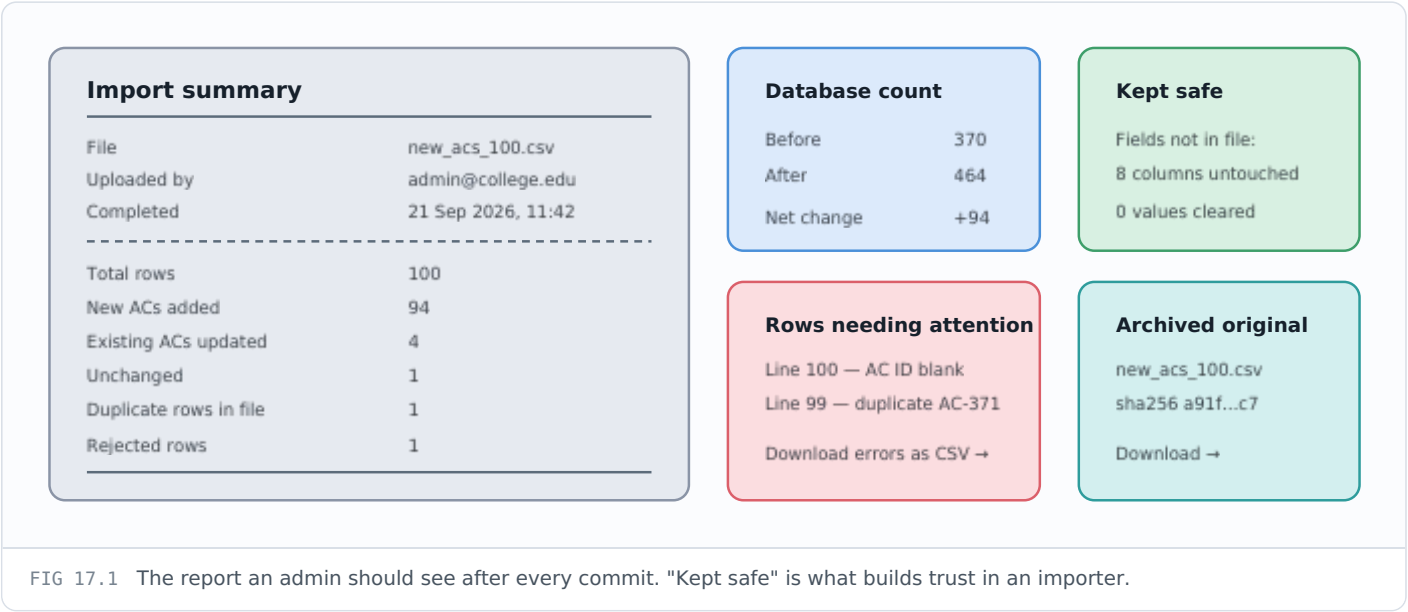
Why repositories are separate

Keeping SQL out of services means you can unit-test reconciliation logic with plain objects and no database at all. That is the part of the system most worth testing.

17 Audit trail and reporting

After a commit, never just say "Upload successful." Show what happened, in numbers the

admin can check against their own spreadsheet.



Change log

For a management system that will be audited, record every field-level change to an AC, whether it came from an import or a manual edit.

```
CREATE TABLE ac_change_log (  
  log_id      BIGSERIAL PRIMARY KEY,  
  ac_id       VARCHAR(50) NOT NULL,  
  field_name  VARCHAR(50) NOT NULL,  
  old_value   TEXT,  
  new_value   TEXT,  
  source      VARCHAR(20) NOT NULL, -- 'import' | 'manual' | 'system'  
  import_id   INT REFERENCES staging_imports(import_id),  
  changed_by  INT,  
  changed_at  TIMESTAMPTZ DEFAULT now()  
);  
  
CREATE INDEX idx_change_ac  ON ac_change_log(ac_id, changed_at DESC);  
CREATE INDEX idx_change_imp ON ac_change_log(import_id);
```

Write these rows from the committer, using the `changed_fields` array that reconciliation already computed — the work is done, it just needs recording. With this table you can answer "who changed AC-045 to Under Maintenance, and which file did it come from?" in one query.

18Test checklist

These are the cases that decide whether the importer is trustworthy. Each should be a fixture CSV and an assertion.

#	Scenario	Expected result
1	Empty database, upload 370 ACs	370 inserted, 0 updated
2	Upload 100 entirely new ACs	470 total, original 370 untouched
3	Re-upload the same 100-row file	470 total, 100 unchanged, 0 writes
4	File with only <code>ac_id</code> , <code>current_status</code>	Only status changes; 12 other columns keep their values
5	Column present, cell blank, option off	Existing value survives
6	Column present, cell blank, option on	Value set to NULL, logged
7	Same <code>ac_id</code> on two lines	Last wins, earlier flagged duplicate
8	Serial number already used by another AC	Row rejected with <code>E_SERIAL_TAKEN</code> , rest imports
9	<code>room_id</code> not in the database	Row rejected, import still commits the valid rows
10	Constraint violation mid-commit	Full rollback, database unchanged, import marked failed
11	Headers in mixed case with spaces	Mapped correctly through aliases
12	Unknown extra column in the file	Ignored, listed in the report, import proceeds
13	File with no <code>ac_id</code> column	Whole file rejected before staging
14	Decommissioned AC appears in a new file	Updated in place; history and maintenance records intact

Build order

If you are starting from nothing, this sequence gets you a working system fastest without painting yourself into a corner:

1. Migrations for the core schema, seeded with one college, a block, a floor and a few rooms.
2. Read endpoints for ACs, so you can see what is in the database.
3. Parser and header mapping, tested against three deliberately messy CSVs.
4. Staging tables and the upload endpoint. Stop and check the JSONB rows look right.
5. Validator with the error-code table from section 14.
5. Reconciler as pure functions, unit tested against fixtures — no database.
7. Committer with the dynamic upsert, then run the whole checklist above.
3. Preview UI, import report, change log.
9. Dashboard and exports last. They are easy once the data is trustworthy.

The one thing to get right

Everything in this document exists to protect a single behaviour: an upload adds and corrects, and never silently destroys. Build step 6 carefully and the rest follows.